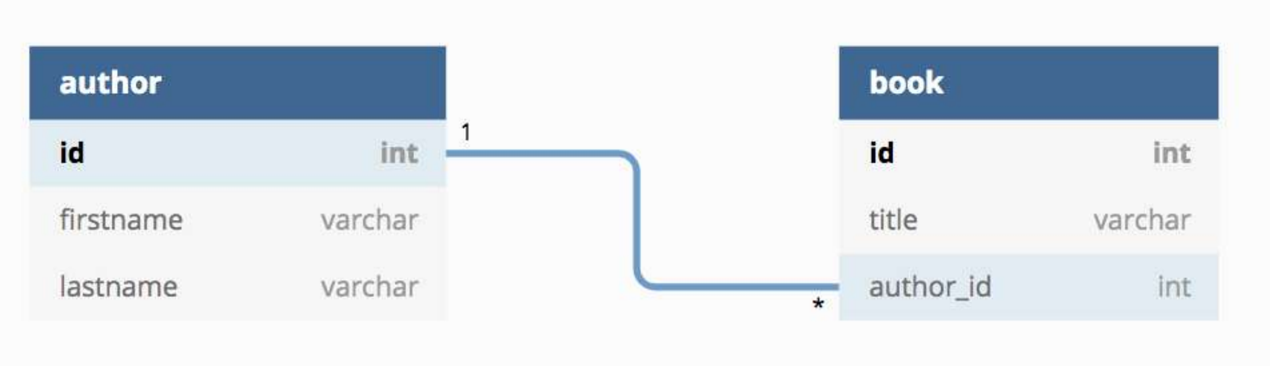
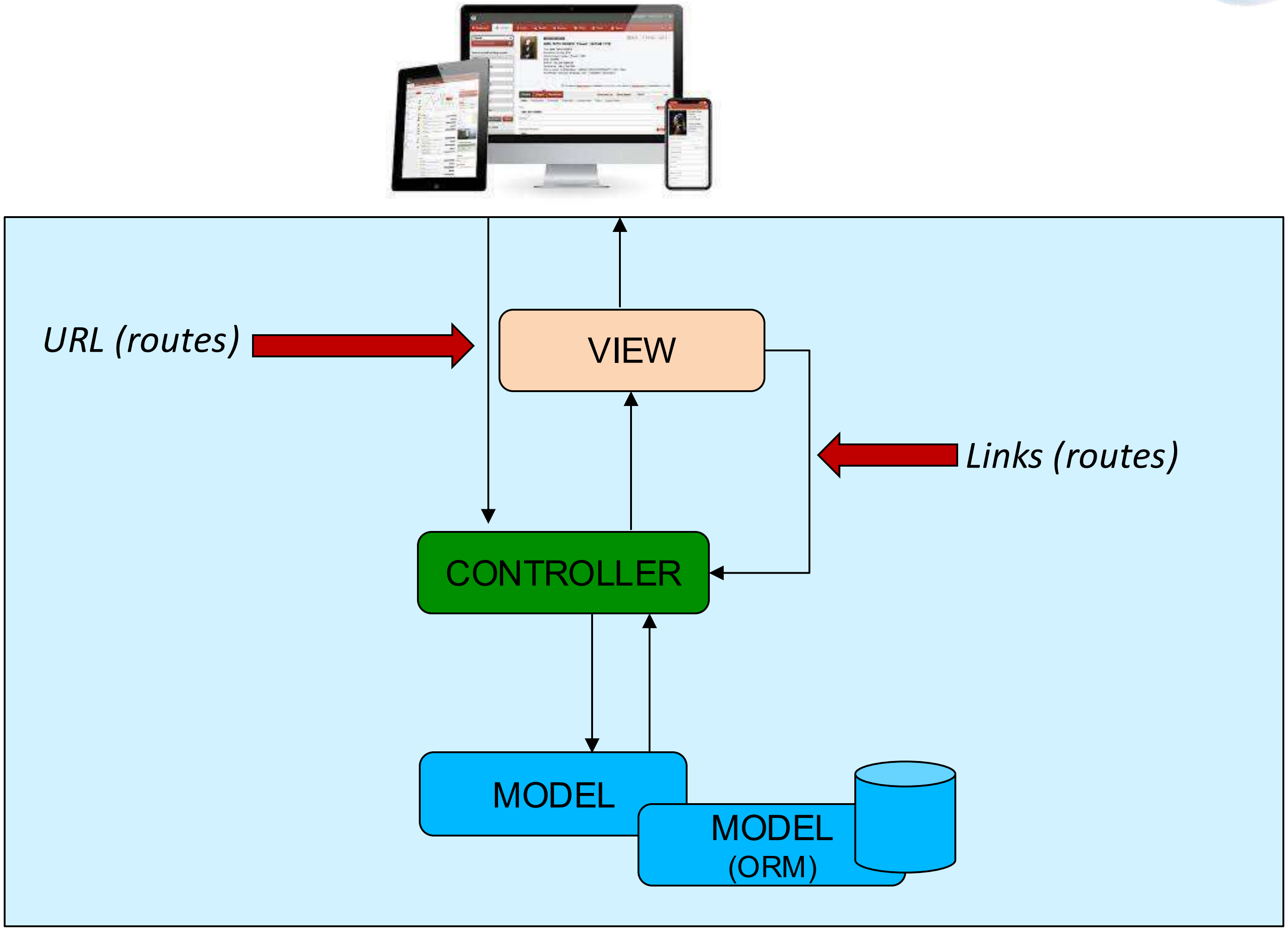
Programmazione Web e Servizi Digitali

Object-Relational Mapping Eloquent

Il database


author
id int
firstname varchar
lastname varchar
1
*
book
id int
title varchar
author_id int

MVC design pattern



Creazione del Model (I)

- ✓ La componente `Model` include tutte le classi PHP in cui viene codificata la logica di business dell'applicazione, così come le classi per l'interazione con il database (ORM)
- ✓ Preparazione dei parametri di configurazione del database (nel file `.env`)

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=8889
DB_DATABASE=MyLibraryDB
DB_USERNAME=devis
DB_PASSWORD=bianchini
```

Creazione del Model (II)

```
php artisan make:model Model
```

Crea un file **`Model.php`** all'interno della cartella **`app/`**

Per convenzione, corrisponde ad una tabella nel database il cui nome è ottenuto da quello del modello:

- ✓ Trasformato da CamelCase a snake_case e pluralizzato (**`MyAuthor`** -> **`my_authors`**)
- ✓ A meno che non si usi la proprietà **`$table`**

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    //protected $table = 'alter_name_for_the_book_table';
}
```

Creazione del Model (III)

Per convenzione, la chiave primaria:

- ✓ È associata al campo **`id`** autoincrementale e intero
- ✓ A meno che non si usi la proprietà **`$primaryKey`**
- ✓ Ogni oggetto della classe **`.php`** generata presenta implicitamente una serie di campi corrispondenti alle colonne della tabella corrispondente nel DB
- ✓ Altre opzioni disponibili (**`id`** non incrementale, non intero, etc.)

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    //protected $table = 'alter_name_for_the_book_table';
    //protected $primaryKey = 'alter_field_as_primary_key';
}
```

Creazione del Model (IV)

- ✓ Per default, Eloquent si aspetta in ogni tabella del database due colonne aggiuntive `created_at` e `updated_at` per tenere traccia delle firme temporali di creazione e modifica di ogni istanza
- ✓ A meno che non si usi la proprietà `public $timestamps = false`
- ✓ Altre opzioni disponibili (come la possibilità di cambiare il nome delle colonne che dovrebbero contenere le firme temporali di creazione e di modifica)
- ✓ Esiste anche la proprietà `deleted_at` per implementare il cosiddetto «soft delete»
  - ✓ Per abilitare questa opzione nel modello va esplicitato `use SoftDeletes`

Relazioni uno-a-uno

Modellata tramite i metodi **`hasOne()`** e **`belongsTo()`**

```
// Method of Author model
0 references | 0 overrides
public function address()
{
    // the property $author->address returns an object of type Address
    return $this->hasOne(Address::class,'author_id','id');
}
// Method of Address model
0 references | 0 overrides
public function author()
{
    // the property $address->author returns an object of type Author
    return $this->belongsTo(Author::class,'author_id','id');
}
```

La convenzione richiede una colonna **`author_id`** nella tabella **`addresses`**, in tal caso il secondo e il terzo parametro dei metodi sono evitabili

Relazioni uno-a-molti

Modellata tramite i metodi **`hasMany()`** e **`belongsTo()`**

```
// Method of Author model
0 references | 0 overrides
public function books()
{
    // the property $author->books returns an array of Books
    return $this->hasMany(Book::class,'author_id','id');
}

// Method of Book model
0 references | 0 overrides
public function author()
{
    // the property $book->author returns an object of type Author
    return $this->belongsTo(Author::class,'author_id','id');
}
```

La convenzione richiede una colonna **`author_id`** nella tabella **`books`**, che punta alla colonna id della tabella authors, in tal caso il secondo e il terzo parametro dei metodi sono evitabili

Relazioni molti-a-molti

Modellata tramite il metodo **`belongsToMany()`**

```
// Method of Book model
5 references | 0 overrides
public function categories()
{
    // the property $book->categories returns an array of Category
    return $this->belongsToMany(Category::class,'book_category','book_id','category_id');
}

// Method of Category model
0 references | 0 overrides
public function books()
{
    // the property $category->books returns an array of Book
    return $this->belongsToMany(Book::class,'book_category','category_id','book_id');
}
```

La convenzione richiede una tabella **`book_category`** con le due colonne **`book_id`** e **`category_id`**; in tal caso, i parametri aggiuntivi possono essere omessi

dentro eloquent non ho bisogno di creare un nuovo modello, ma devo mettere il doppio rifermimento come mostrato qui sopra

Creazione delle tabelle nel DB (I)

Laravel offre il meccanismo delle *migration* – Si tratta di script che sono posizionati nella cartella **`database/migrations/`** e si occupano del versioning del database tramite i metodi **`up()`** e **`down()`**

```
php artisan make:migration book_table
```

A questo punto, il comando `php artisan migrate` genererà le tabelle nel DB
Per annullare le migration eseguite sull'applicazione il comando da utilizzare è `php artisan migrate:rollback`

1. cosa sono i seeder
2. popolazioni automatiche massive

Creazione delle tabelle nel DB (II)

```php
Schema::create('author', function (Blueprint $table) {
    $table->id();
    $table->string('firstname');
    $table->string('lastname');
    $table->timestamps();
});

Schema::create('book', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->unsignedBigInteger('author_id');
    $table->timestamps();
});
```

```php
Schema::table('book', function (Blueprint $table) {
    $table->foreign('author_id')->references('id')->on('author');
});
```

Popolamento delle tabelle nel DB

Laravel offre anche gli strumenti (denominati *Factory* e *Seed*) per popolare le tabelle del DB con dati di test

- Le factory sono classi che si occupano di costruire istanze di particolari modelli inserendo dati di test nelle loro proprietà (cartella **`database/factories`**)
- I seed sono script per il popolamento del database (che possono far uso delle factory) attraverso il comando `php artisan make:seeder ${nome_seeder}`, che genererà uno script `${nome_seeder}` nella cartella **`database/seeds`**; il seeder viene invocato con il comando `php artisan db:seed --class=${nome_seeder}`

Factories - Esempio

```
php artisan make:factory BookFactory [-model=Book]
```

```php
namespace Database\Factories;

use App\Models\Book;
use Illuminate\Database\Eloquent\Factories\Factory;

/**
 * @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\Book>
 */
0 references | 0 implementations
class BookFactory extends Factory
{
    0 references
    protected $model = Book::class;
    0 references | 0 overrides
    public function definition(): array
    {
        return [
            'title' => $this->faker->sentence(nbWords: rand(min: 1, max: 5))
        ];
    }
}
```

```php
class Book extends Model
{
    use HasFactory;
```

Seeders - Esempio

```
php artisan make:seeder DatabaseSeeder
```

```
public function run(): void
{
    // User::factory(10)->create();

    //User::factory()->create([
    //    'name' => 'Test User',
    //    'email' => 'test@example.com',
    //]);
    $this->populateDB();
}

1 reference
private function populateDB(): void
{

    // Create 100 authors
    Author::factory()->count(100)->create();

    // Randomly select a subset of 50 authors and, for each of them, create a set of books
    $authors = Author::all();
    $authorsWithBooks = $authors->random(number: 50);

    foreach($authorsWithBooks as $singleAuthor) {
        $numberOfBooks = rand(min: 1,max: 5);
        for($b=0; $b<$numberOfBooks; $b++) {
            Book::factory()->count(1)->create(['author_id' => $singleAuthor->id]);
        }
    }
}
```

Per l'inserimento massivo…

Proprietà modificabili in modalità *massiva* (o non modificabili):

```
class Book extends Model
{
    use HasFactory;
    0 references
    protected $table = "book";
    // protected $primaryKey = 'alter_field_as_primary_key';
    // use SoftDeletes;
    // public $timestamps = false;

    0 references
    protected $fillable = ['title', 'author_id'];

    // Method of Book model
    0 references | 0 overrides
    public function author(): BelongsTo
    {
        // the property $book->author returns an object of type Author
        return $this->belongsTo(related: Author::class,foreignKey: 'author_id',ownerKey: 'id');
    }
}
```

Tutto insieme!

```
php artisan make:model Model [opzioni]
```

Migration, factories, seeder e controller possono anche essere creati insieme al modello:

- `-c,--controller`: genera anche un controller per il modello
- `-r,--resource` (solo insieme a `-c`): genera un controller «resourceful»
- `-f,--factory`: genera anche una factory per il modello
- `-m,--migration`: genera anche una migration per il modello
- `-s,--seed`: genera anche un seeder per il modello
- `-a,--all`

DataLayer con Eloquent (I)

```
public function listBooks(): Collection
{
    $booksList = Book::orderBy(column: 'title',direction: 'asc')->get();
    return $booksList;
}
```

```
public function listAuthors(): Collection
{
    $authorsList = Author::orderBy(column: 'lastname',direction: 'asc')
    ->orderBy(column: 'firstname',direction: 'asc')->get();
    return $authorsList;
}
```

DataLayer con Eloquent (II)

```
public function findBookById($id): Collection
{
    return Book::find(id: $id);
}
```

```
public function findAuthorById($id): Collection
{
    return Author::find(id: $id);
}
```

DataLayer con Eloquent (III)

```php
public function addBook($title,$author_id): void
{
    $book = new Book;
    $book->title = $title;
    $book->author_id = $author_id;
    $book->save();
    // massive creation (only with fillable property enabled on Book):
    // Book::create(['title' => $title, 'author_id' => $author_id]);
}
```

```php
public function addAuthor($first_name,$last_name): void
{
    $author = new Author;
    $author->firstname = $first_name;
    $author->lastname = $last_name;
    $author->save();
    // massive creation (only with fillable property enabled on Author):
    // Author::create(['firstname' => $first_name, 'lastname' => $last_name]);
}
```

DataLayer con Eloquent (IV)

```php
public function editBook($id,$title,$author_id): void
{
    $book = Book::find(id: $id);
    $book->title = $title;
    $book->author_id = $author_id;
    $book->save();
    // massive update (only with fillable property enabled on Book):
    // Book::find($id)->update(['title' => $title, 'author_id' => $author_id]);
}
```

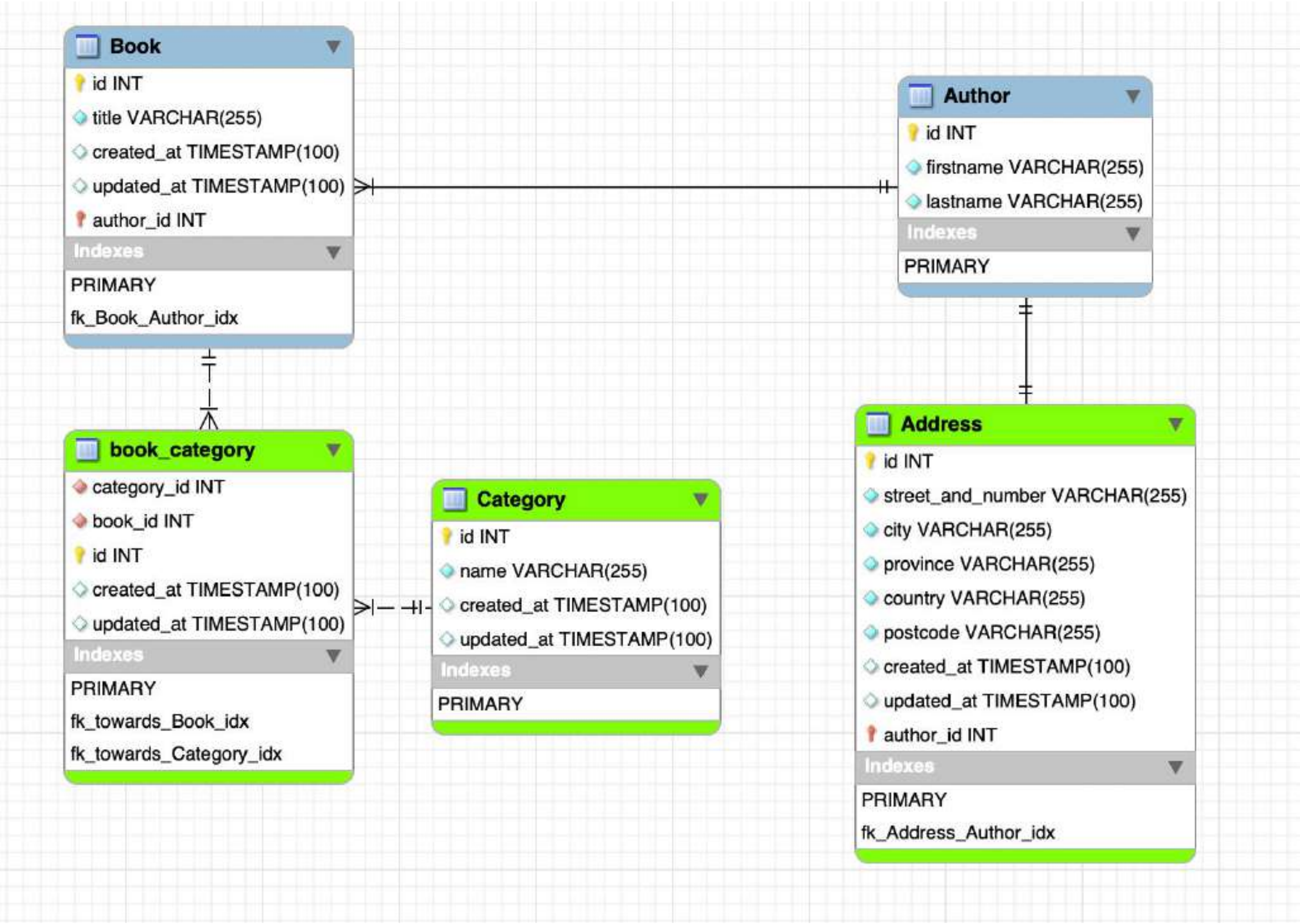
```php
public function editAuthor($id,$first_name,$last_name): void
{
    $author = Author::find(id: $id);
    $author->firstname = $first_name;
    $author->lastname = $last_name;
    $author->save();
    // massive update (only with fillable property enabled on Author):
    // Author::find($id)->update(['firstname' => $first_name, 'lastname' => $last_name]);
}
```

DataLayer con Eloquent (V)

```
public function deleteBook($id): void
{
    $book = Book::find(id: $id);
    $book->delete();
}
```

```
public function deleteAuthor($id): void
{
    $author = Author::find(id: $id);
    $author->delete();
}
```

DB “extended”


Book
id INT
title VARCHAR(255)
created_at TIMESTAMP(100)
updated_at TIMESTAMP(100)
author_id INT
Indexes
PRIMARY
fk_Book_Author_idx
Author
id INT
firstname VARCHAR(255)
lastname VARCHAR(255)
Indexes
PRIMARY
book_category
category_id INT
book_id INT
id INT
created_at TIMESTAMP(100)
updated_at TIMESTAMP(100)
Indexes
PRIMARY
fk_towards_Book_idx
fk_towards_Category_idx
Category
id INT
name VARCHAR(255)
created_at TIMESTAMP(100)
updated_at TIMESTAMP(100)
Indexes
PRIMARY
Address
id INT
street_and_number VARCHAR(255)
city VARCHAR(255)
province VARCHAR(255)
country VARCHAR(255)
postcode VARCHAR(255)
created_at TIMESTAMP(100)
updated_at TIMESTAMP(100)
author_id INT
Indexes
PRIMARY
fk_Address_Author_idx

DB “extended” - Seeders (I)

```
class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    0 references | 0 overrides
    public function run(): void
    {
        $this->populateDB();
    }

    1 reference
    private function populateDB(): void
    {

        // Create 100 authors with their corresponding address
        Author::factory()->count(100)->create()->each(function ($author): void {
            Address::factory()->count(1)->create(['author_id' => $author->id]);
        });

        // Randomly select a subset of 50 authors and, for each of them, create a set of books
        $authors = Author::all();
        $authorsWithBooks = $authors->random(number: 50);

        foreach($authorsWithBooks as $singleAuthor) {
            $numberOfBooks = rand(min: 1,max: 5);
            for($b=0; $b<$numberOfBooks; $b++) {
                Book::factory()->count(1)->create(['author_id' => $singleAuthor->id]);
            }
        }
```

DB “extended” - Seeders (II)

```php
    // Create 10 book categories
    Category::factory()->count(1)->create(['name' => 'Romanzi classici']);
    Category::factory()->count(1)->create(['name' => 'Fantasy']);
    Category::factory()->count(1)->create(['name' => 'Gialli']);
    Category::factory()->count(1)->create(['name' => 'Thriller']);
    Category::factory()->count(1)->create(['name' => 'Saggi']);
    Category::factory()->count(1)->create(['name' => 'Poesie']);
    Category::factory()->count(1)->create(['name' => 'Psicologia']);
    Category::factory()->count(1)->create(['name' => 'Fantascienza']);
    Category::factory()->count(1)->create(['name' => 'Viaggi']);
    Category::factory()->count(1)->create(['name' => 'Arte e fotografia']);

    // Randomly associate a book to a subset of categories (from 1 to 5)
    $books = Book::all();
    $categories = Category::all();

    foreach($books as $singleBook)
    {
        $numberOfCategories = rand(1,5);
        $selectedCategories = $categories->random($numberOfCategories);
        $singleBook->categories()->attach($selectedCategories);
    }
}
```

DB “extended” – DataLayer (I)

```
1 reference | 0 overrides
public function addBook($title, $author_id, $categories) {
    $book = new Book;
    $book->title = $title;
    $book->author_id = $author_id;
    $book->save();
    foreach($categories as $cat) {
        $book->categories()->attach($cat);
    }
    // massive creation (only with fillable property enabled on Book):
    // Book::create(['title' => $title, 'author_id' => $author_id, 'user_id' => $user]);
}

0 references | 0 overrides
public function addAuthor($first_name, $last_name) {
    $author = new Author;
    $author->firstname = $first_name;
    $author->lastname = $last_name;
    $author->save();

    //use the factory to randomly generate an address
    Address::factory()->count(1)->create(['author_id' => $author->id]);

    // massive creation (only with fillable property enabled on Author):
    // Author::create(['firstname' => $first_name, 'lastname' => $last_name, 'user_id' => $user]);
}
```

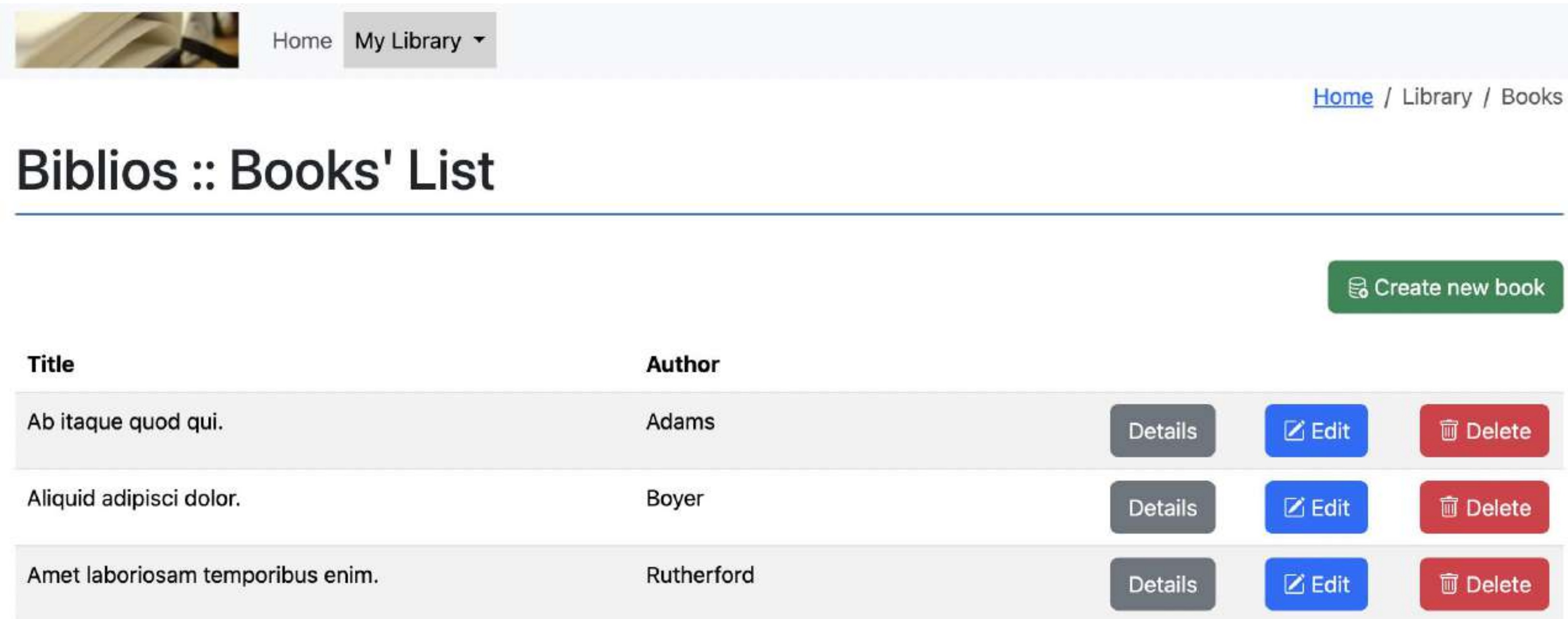
DB “extended” – DataLayer (II)

```
1 reference | 0 overrides
public function editBook($id, $title, $author_id, $categories) {
    $book = Book::find($id);
    $book->title = $title;
    $book->author_id = $author_id;
    $book->save();

    // Cancel the previous list of categories
    $prevCategories = $book->categories;
    foreach($prevCategories as $prevCat) {
        $book->categories()->detach($prevCat->id);
    }

    // Update the list of categories
    foreach($categories as $cat) {
        $book->categories()->attach($cat);
    }
    // massive update (only with fillable property enabled on Book):
    // Book::find($id)->update(['title' => $title, 'author_id' => $author_id]);
}

0 references | 0 overrides
public function editAuthor($id, $first_name, $last_name) {
    $author = Author::find($id);
    $author->firstname = $first_name;
    $author->lastname = $last_name;
    $author->save();
    // massive update (only with fillable property enabled on Author):
    // Author::find($id)->update(['firstname' => $first_name, 'lastname' => $last_name]);
}
```

DB “extended” – DataLayer (III)

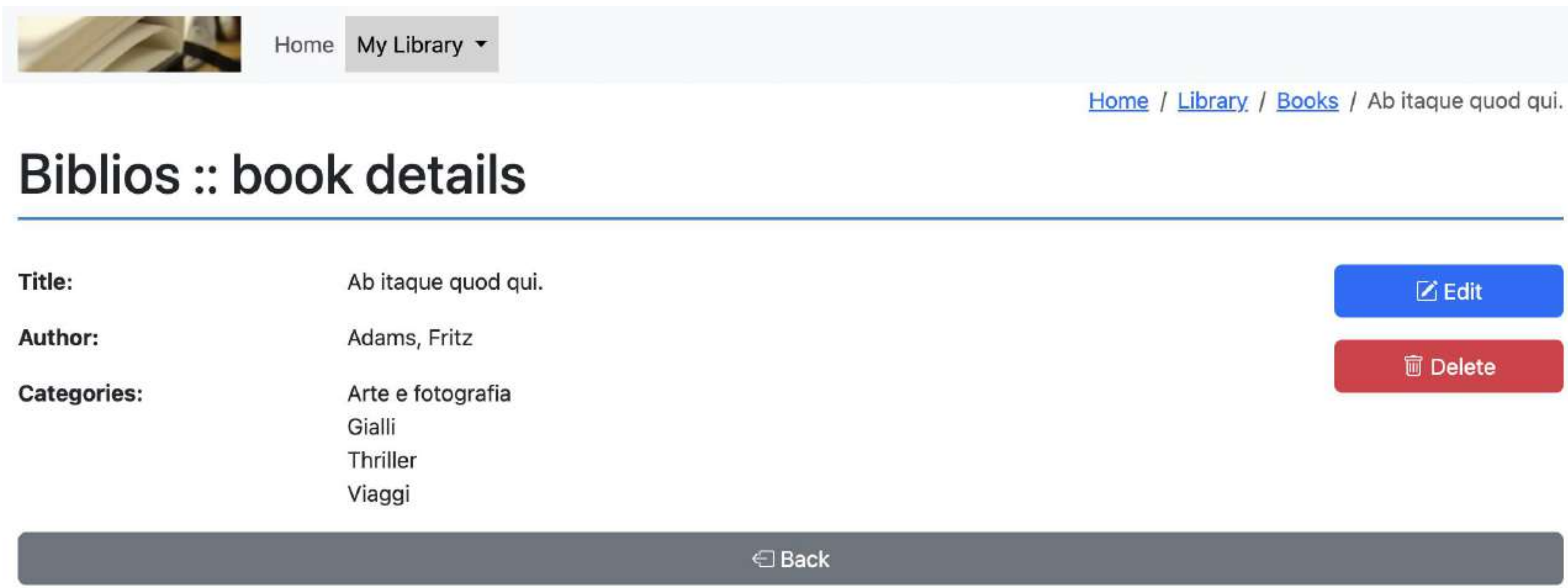
```
1 reference | 0 overrides
public function deleteBook($id) {
    $book = Book::find($id);
    $categories = $book->categories;
    foreach($categories as $cat) {
        $book->categories()->detach($cat->id);
    }
    $book->delete();
}

0 references | 0 overrides
public function deleteAuthor($id) {
    $author = Author::find($id);
    $author->address->delete();
    $author->delete();
}
```

Running example v6-plus (I)



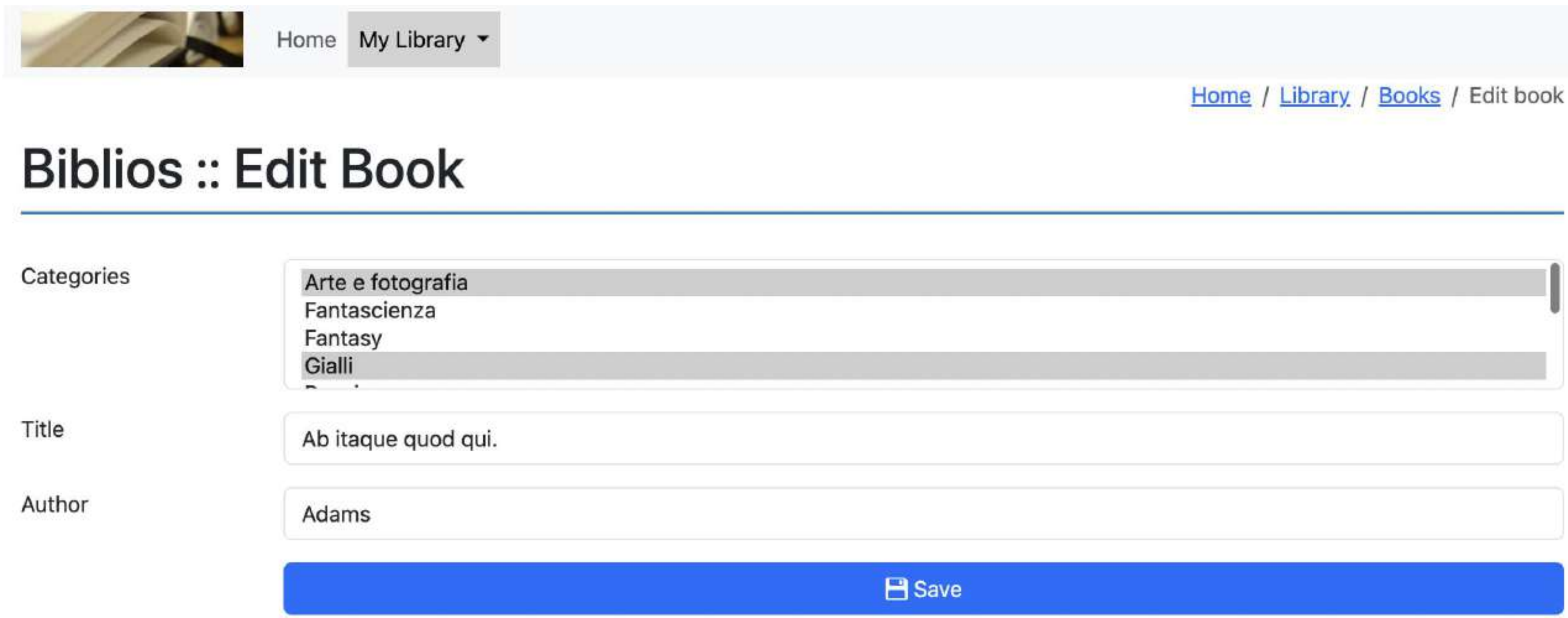

```
Route::get('/book', [BookController::class, 'index'])->name('book.index'); // Display the list of books
```

Running example v6-plus (II)




```
Route::get('/book/{id}', [BookController::class, 'show'])->name('book.show'); // Display a single book
```

Running example v6-plus (III)




```
Route::get('/book/{id}/edit', [BookController::class, 'edit'])->name('book.edit'); // Display the edit form
```

```
@if(isset($book->id))
    <form class="form-horizontal" name="book" method="post" action="{{ route('book.update', ['book' => $book->id]) }}">
    <!--<input type="hidden" name="_method" value="PUT">-->
    @method('PUT')
@else
    <form class="form-horizontal" name="book" method="post" action="{{ route('book.store') }}">
@endif
@csrf
```

Link utili

La documentazione ufficiale di Eloquent inclusa in quella di Laravel: https://laravel.com/docs/11.x/eloquent

Programmazione Web e Servizi Digitali

Object-Relational Mapping Eloquent